

Software Requirements Document (SRD)

Project: Radar - Stealth Activity & Network Intelligence

1. Product Overview

Radar is an advanced, ultra-stealth background service designed to run 24/7 on a host laptop. Its primary objective is the seamless, undetectable monitoring of both the host's daily activities (foreground applications, terminal commands) and the external network environment.

Crucially, Radar acts as a passive network sentinel. It not only tracks the host machine but extends its surveillance to all other devices connected to the same WiFi network, performing reconnaissance without triggering security alerts. At the end of each day, Radar processes this intelligence into a comprehensive, discreet daily report and emails it directly to the user's Gmail.

2. Functional Requirements

2.1 Ultra-Stealth & Performance (Zero-Footprint)

- The system must operate completely invisibly (no tray icons, no visible processes).
- It must maintain an ultra-low performance footprint (< 2% CPU usage, minimal RAM) to remain undetected.

2.2 Host Activity Monitoring

- Track open applications, window focus sequence, and exact time spent.
- Silently record shell and terminal histories along with timestamps.

2.3 Advanced Network Surveillance (Other Devices)

- Passively monitor the local WiFi network.
- Identify and track every other device connected to the network via MAC mapping and OS fingerprinting.
- Monitor broader network performance and intercept broadcast/multicast telemetry to map device activity.

2.4 Automated Intelligence Reporting

- Nightly aggregation of all intelligence into an HTML/Text email payload.
- Automatically attach a separate text log capturing raw network dumps and tracking data of other devices.

3. System Architecture & Flow

Phase 1: Ghost Initialization

The system registers via hidden system hooks (e.g., systemd/launchd with obfuscated daemon names). It establishes a localized, encrypted SQLite database for resilient data buffering.

Phase 2: Silent Polling & Sniffing

The host application monitor runs at low-priority intervals to avoid CPU spikes. Concurrently, the network module periodically drops into promiscuous-like states or uses silent ARP requests to seamlessly inventory

Software Requirements Document (SRD)

Project: Radar - Stealth Activity & Network Intelligence

network peers.

Phase 3: Exfiltration Payload

At a randomized or scheduled time during the night, the payload builder formats the daily intelligence report and securely tunnels it via SMTP (TLS) to the destination Gmail address.

4. Payload Deliverables

Daily Intelligence Email Content:

- Host Analytics Table: Top applications used with duration, and complete explicit terminal command history.
- Network Intelligence Summary: Snapshot of the local WiFi ecosystem and status of peer devices.
- Attached Log (network_intel_YYYYMMDD.txt): Detailed raw records outlining network status and peer device telemetry throughout the day.

Software Requirements Document (SRD)

Project: Radar - Stealth Activity & Network Intelligence

5. Implementation Strategy & Libraries

The project will be engineered in Python 3.11+ using the following structure:

Core Libraries:

- 'psutil': Absolute low-impact process and active-window monitoring.
- 'scapy': Advanced packet crafting and network reconnaissance (ARP/DNS tracking).
- 'schedule': Highly reliable scheduling for report triggers.
- 'smtplib' & 'email': Built-in tools for securely compiling and dispatching payloads via Gmail SMTP.
- 'sqlite3': Built-in database module operating in WAL mode to avoid write-locks and ensure data integrity.

File Structure:

- core_daemon.py: The obfuscated main loop.
- monitors/host_tracker.py: Application and terminal history scraping.
- monitors/net_sentinel.py: WiFi scanning and peer device MAC tracking.
- reporting/payload_builder.py: Formats and prepares data for exfiltration.
- database/vault.py: Interacts with the local SQLite data store safely.

6. Software Testing Strategy

Absolute reliability and stealth are non-negotiable. The testing regimen includes:

1. Unit Testing ('pytest'):

Modules like the host_tracker and payload_builder will be tested in complete isolation using 'pytest-mock'. This simulates system APIs without actually triggering or disrupting the host OS during CI/CD checks.

2. Performance & Stealth Profiling:

The daemon will be stress-tested using performance profilers to mathematically guarantee it never spans above 2% CPU utilization or 50MB of RAM. Persistent memory leak checks will be enforced over 72-hour sustained runs.

3. E2E (End-to-End) Exfiltration Tests:

Sandboxed environments will validate the full lifecycle - from data injection into the SQLite vault, to the successful receipt of the HTML/Text payload in a sandbox Gmail inbox without triggering spam filters.

4. Signature & Obfuscation Audits:

Continuous monitoring against standard task managers to verify process name obfuscation is successful and execution remains entirely invisible to a standard user.